SAECOIN

DATA SCHEMA SPECIFICATION

Document Version: 1.0
Project: SAECOIN
Ticker: SAC (Provisional)
Blockchain Type: Independent Layer-1
Transaction Model: UTXO
Database: RocksDB (Provisional)
Document Status: Planning

---

1. PURPOSE

This document defines the proposed data structures and database organization for the SAECOIN blockchain ecosystem.

It describes how the system will organize and store:

- Blocks
- Block headers
- Transactions
- Transaction inputs
- Transaction outputs
- UTXO records
- Blockchain metadata
- Wallet data
- Node information
- Mining information
- Explorer indexes

The schema must support blockchain validation, transaction processing, wallet operations, mining, synchronization, and future expansion.

---

2. DATA DESIGN PRINCIPLES

The following principles must be followed:

1. All consensus-critical data must be deterministic.
2. Monetary values must use integer arithmetic.
3. Floating-point values must never be used for blockchain monetary calculations.
4. Consensus data must be separated from local application data.
5. Private keys and seed phrases must never be stored in plaintext.
6. Database updates must support atomic operations where required.

7. Data structures must support future protocol upgrades.
8. Public blockchain data must not contain unnecessary personal information.

---

3. BASIC DATA TYPES

The following data types are proposed.

Type| Description
"u8"| Unsigned 8-bit integer
"u16"| Unsigned 16-bit integer
"u32"| Unsigned 32-bit integer
"u64"| Unsigned 64-bit integer
"u128"| Unsigned 128-bit integer
"i64"| Signed 64-bit integer
"bool"| Boolean
"byte[]"| Binary data
"Hash"| Cryptographic hash
"PublicKey"| Public key
"Signature"| Digital signature
"Address"| Encoded wallet address

Exact field widths and byte order must be finalized in the Consensus Specification.

---

4. BLOCK DATA STRUCTURE

A block contains a header and a collection of transactions.

Proposed Block Structure

Block
│
├── Header
│ ├── Version
│ ├── Previous Block Hash
│ ├── Merkle Root
│ ├── Timestamp
│ ├── Difficulty Target
│ ├── Nonce
│ └── Additional Consensus Fields
│
└── Transactions
 ├── Coinbase Transaction
 ├── Transaction 1
 ├── Transaction 2

```
    └── ...
```

Block Header Fields

| Field | Description |
|---|---|
| Version | Block format version |
| Previous Block Hash | Hash of the parent block |
| Merkle Root | Root hash of block transactions |
| Timestamp | Block creation timestamp |
| Difficulty Target | Target used for PoW validation |
| Nonce | Mining nonce |
| Additional Fields | Any fields required by the finalized consensus protocol |

The exact block header format must be finalized in "Consensus_Spec.md".

Block Metadata

Local node metadata may include:

- Block height.
- Block hash.
- Cumulative chain work.
- Validation status.
- Database location.
- Connection to parent and child blocks.

Local metadata must not be confused with consensus-critical block contents.

---

5. TRANSACTION DATA STRUCTURE

SAECOIN uses a UTXO transaction model.

Proposed Transaction Structure

```
Transaction
│
├── Version
├── Inputs[]
│    ├── Previous Transaction ID
│    ├── Output Index
│    ├── Unlocking Data
│    └── Sequence
│
├── Outputs[]
│    ├── Amount
│    └── Locking Script
```



This is a conceptual structure. The final binary format and signature rules will be defined separately.

Transaction Fields

| Field | Description |
| --- | --- |
| Version | Transaction format version |
| Inputs | Previous outputs being spent |
| Outputs | New outputs being created |
| Lock Time | Optional transaction lock condition |

Transaction Input

An input references an existing unspent output.

Proposed fields:

- Previous transaction ID.
- Previous output index.
- Unlocking script or witness data.
- Sequence number.

Transaction Output

An output creates a new spendable UTXO.

Proposed fields:

- Amount in smallest SAC units.
- Locking script or spending condition.

Monetary Precision

SAECOIN will use 8 decimal places.

1 SAC = 100,000,000 smallest units

All transaction amounts must be stored as integers.

Example:

Amount: 2.50000000 SAC
Stored Value: 250000000

---

6. UTXO DATA STRUCTURE

UTXO means Unspent Transaction Output.

The UTXO set represents all currently unspent transaction outputs.

UTXO Record

UTXO
│
├── Transaction ID
├── Output Index
├── Amount
├── Locking Script
└── Creation Height

UTXO Key

A UTXO can be uniquely identified using:

Transaction ID + Output Index

UTXO Operations

The node must support:

- Add new UTXO.
- Remove spent UTXO.
- Check UTXO existence.
- Validate transaction inputs.
- Calculate available outputs.

The UTXO set is consensus-critical state.

A normal account-balance table must not be used as the authoritative consensus source.

Wallet balances should be calculated from relevant unspent outputs.

---

7. COINBASE TRANSACTION

A coinbase transaction is created by the block-producing miner.

Proposed structure:

CoinbaseTransaction

```
│
├── Transaction Version
├── Coinbase Input
├── Reward Output[]
└── Additional Coinbase Data
```

The coinbase transaction must follow consensus rules.

It may include:

- Block subsidy.
- Transaction fees collected from the block.
- Miner payout output.

The node must verify that the total coinbase reward does not exceed the allowed subsidy plus permitted transaction fees.

The subsidy calculation must enforce the 21,000,000 SAC maximum supply.

Exact reward rules will be finalized in "Consensus_Spec.md".

---

8. BLOCKCHAIN METADATA

The node requires local metadata to manage the active chain and alternative chains.

Chain Metadata

```
ChainMetadata
│
├── Network ID
├── Genesis Block Hash
├── Current Best Block Hash
├── Current Block Height
├── Cumulative Chain Work
└── Database Schema Version
```

Block Index Record

```
BlockIndex
│
├── Block Hash
├── Previous Block Hash
├── Block Height
├── Cumulative Chain Work
├── Validation Status
└── Storage Reference
```

The block index must support:

- Chain synchronization.
- Alternative chain tracking.
- Chain reorganization.
- Block validation status.

---

9. UNDO DATA

Undo data is required to reverse state changes when the node reorganizes its active chain.

Proposed information:

- Removed UTXOs.
- Restored UTXOs.
- Block identifier.
- Related state changes.

Undo data must be associated with the relevant block.

This allows the node to safely disconnect a block and restore the previous UTXO state.

---

10. MEMPOOL DATA STRUCTURE

The mempool stores valid but unconfirmed transactions.

Mempool Record

MempoolTransaction
│
├── Transaction ID
├── Transaction Data
├── Fee
├── Arrival Time
├── Transaction Size
└── Local Status

Possible local statuses:

- Received.
- Validated.
- Waiting.
- Selected for Block.

- Removed.

Mempool data is local operational data, not consensus state.

Each node may have a different mempool.

---

11. WALLET DATA STRUCTURE

Wallet data must be stored separately from blockchain consensus data.

Wallet Record

```
Wallet
│
├── Wallet ID
├── Wallet Name
├── Wallet Version
├── Encrypted Key Material
├── Encryption Metadata
├── Address Records[]
├── Transaction Records[]
└── Wallet Settings
```

Address Record

```
WalletAddress
│
├── Address ID
├── Address
├── Derivation Index
├── Address Label
├── Creation Time
└── Address Type
```

Wallet Transaction Record

```
WalletTransaction
│
├── Transaction ID
├── Transaction Type
├── Amount
├── Fee
├── Transaction Status
├── Block Height
└── Timestamp
```

Wallet transaction records are application-level records.

The wallet must verify relevant blockchain data before presenting confirmed transaction information.

Wallet Security

- Seed phrases must not be stored in plaintext.
- Private keys must be encrypted at rest.
- Encryption metadata must identify the encryption format and version.
- Wallet passwords must never be stored directly.
- Sensitive data must not appear in logs.
- Wallet backup and recovery must be supported.

---

12. NODE DATA STRUCTURE

Node information is stored locally.

Node Configuration

NodeConfig
│
├── Network ID
├── Data Directory
├── Listening Port
├── Maximum Peer Connections
├── Sync Settings
└── Node Version

Peer Record

PeerRecord
│
├── Peer Address
├── Peer Port
├── Protocol Version
├── Last Seen
├── Connection Status
└── Peer Score

Peer records are operational metadata.

They must not influence consensus validity.

---

13. MINING DATA STRUCTURE

Mining information is separated from consensus-critical blockchain data.

Mining Job

MiningJob
│
├── Job ID
├── Previous Block Hash
├── Candidate Block Data
├── Difficulty Target
├── Creation Time
└── Job Status

Miner Information

MinerRecord
│
├── Miner ID
├── Worker Name
├── Mining Mode
├── Hashrate
├── Last Share Time
└── Connection Status

Hashrate and worker information are operational statistics.

They do not determine whether a block is valid.

Mining Pool Data

If mining pools are implemented, their separate database may contain:

- Pool accounts.
- Worker records.
- Submitted shares.
- Share validation results.
- Payout records.
- Pool statistics.

Mining pool records must never be treated as blockchain consensus data.

---

14. EXPLORER DATA STRUCTURE

The block explorer requires indexes for fast searches.

Explorer Block Index

ExplorerBlock
│
├── Block Height
├── Block Hash
├── Timestamp
├── Transaction Count
├── Block Size
├── Difficulty
└── Miner Address (if identifiable)

Explorer Transaction Index

ExplorerTransaction
│
├── Transaction ID
├── Block Hash
├── Block Height
├── Transaction Size
├── Input Count
├── Output Count
└── Timestamp

Explorer Address Index

ExplorerAddress
│
├── Address
├── Related Transactions[]
└── Indexed Output References[]

Explorer indexes are derived data.

They can be rebuilt from validated blockchain data and must not replace the authoritative blockchain state.

---

15. DATABASE ORGANIZATION

RocksDB is the proposed initial key-value storage engine.

The database may use logical key prefixes.

Proposed prefixes:

Prefix| Data
"block:"| Serialized block data
"blockindex:"| Block index records
"tx:"| Transaction records or indexes
"utxo:"| UTXO records
"undo:"| Block undo data
"chain:"| Chain metadata
"mempool:"| Local mempool records
"peer:"| Peer metadata
"schema:"| Database schema version

The exact key encoding must be documented before implementation.

Database writes affecting a block's state must be coordinated safely so that partial updates cannot leave the active chain and UTXO set inconsistent.

Wallet, explorer, and mining-pool databases should remain logically separate from consensus-critical node storage.

---

16. DATA SERIALIZATION

Consensus-critical data must use deterministic serialization.

Requirements:

- Every field must have a defined encoding.
- Field ordering must be fixed.
- Integer encoding must be specified.
- Byte order must be specified.
- Variable-length fields must have defined length encoding.
- Invalid or oversized data must be rejected.
- Serialization must be consistent across supported platforms.

JSON may be used for configuration files, APIs, and debugging.

JSON must not automatically be treated as the canonical consensus serialization format.

The final binary serialization specification will be defined in "Consensus_Spec.md".

---

17. SCHEMA VERSIONING

The system must support schema versioning.

Requirements:

- Database schema version.
- Wallet data format version.
- Transaction format version.
- Block format version.
- Network protocol version.

Database upgrades must include migration procedures where necessary.

A node must not silently interpret incompatible data using an incorrect schema.

---

18. DATA VALIDATION

All incoming data must be validated.

Validation must include:

- Required field checks.
- Data length checks.
- Integer range checks.
- Hash format checks.
- Transaction structure checks.
- Signature verification.
- UTXO existence checks.
- Block structure checks.
- Serialization checks.

Malformed data must be rejected safely.

---

19. CONSENSUS-CRITICAL VS NON-CONSENSUS DATA

The system must clearly separate the two categories.

Consensus-Critical Data

- Block header.
- Block transactions.
- Transaction inputs and outputs.
- Transaction signatures or witness data.
- UTXO state.
- Subsidy calculation.
- Difficulty and PoW validation data.
- Chain selection data required by consensus.

Non-Consensus Data

- Wallet labels.
- Wallet display preferences.
- Peer connection history.
- Explorer search indexes.
- Mining dashboard statistics.
- Pool worker information.
- Local node logs.
- UI settings.

Changes to non-consensus data must not alter blockchain validity.

---

20. FUTURE EXPANSION

The schema should allow future support for:

- Additional wallet types.
- Hardware wallet integration.
- Advanced transaction scripts.
- Multiple mining implementations.
- Additional explorer indexes.
- Improved node synchronization.
- Future protocol upgrades.

Future additions must not compromise existing consensus rules.

---

21. FINAL NOTES

This document defines the proposed data model for SAECOIN.

The exact binary layouts, cryptographic hash functions, signature algorithms, field widths, transaction limits, block limits, and consensus serialization rules are not finalized here.

These must be specified and reviewed in the dedicated documents:

- "Consensus_Spec.md"
- "Blockchain_Architecture.md"
- "Security_Spec.md"
- "API_Spec.md"

The database engine and physical storage layout remain subject to testing and benchmarking.

END OF Schema.md